User Modeling and User-Adapted Interaction manuscript No.
(will be inserted by the editor)

A Model for Profile Management Through Inference in Trails

André Wagner · Jorge Luis Victória Barbosa · Débora Nice Ferrari Barbosa

Received: date / Accepted: date

Abstract Context-aware and ubiquitous systems have the challenge of implicitly collect relevant information about the users, and use this information to understand and predict their behaviour. This allows the applications to adapt themselves to the user, thus avoiding to overflow the user with inquires and information. This article presents a model that allows applications to register users' actions and infer profile information from these actions, using semantic interoperability and thus allowing different applications of different domains to be able to share information and infer a unified profile.

Keywords Profile Management · Ubiquitous Computing · Trails · Profile Inference

1 Introduction

Nowadays, most users interact with a wide variety of computing systems. The current tendency is the increased interconnectedness between these systems, since the internet is becoming every day more present and available. Recent research predicts the full integration between these systems in a federation of computing systems. This federation is referred to as Smart Environment (Heckmann, 2005).

André Wagner · Jorge Luis Victória Barbosa
Universidade do Vale do Rio dos Sinos (UNISINOS)
Unidade Acadêmica de Pesquisa e Pós-Graduação
Av. Unisinos, 950, São Leopoldo, Brazil
E-mail: andre.nho@gmail.com, jbarbosa@unisinos.br

Débora Nice Ferrari Barbosa
Universidade Feevale
Gr. Pesquisa em Informática na Educação
RS-239, 2755, Novo Hamburgo, Brazil
E-mail: deboranice@feevale.br

Weiser (1991) discussed a vision for the concept of smart envirnoments. He presented a scenario in which the computer would become invisible and indistinguishable from its environment. In this scenario, interaction between humans and computers would not occur as usual – where the humans devote their full atention to the computer –, but would consist of small and quick interactions, usually in different devices and locations, and in many times using the "periphery" of the user's attention (Weiser & Brown, 1996), or without the user even realizing he is interacting with a computer system. According to the author, "the most profound technologies are those that disappear. They weave themselves into the fabric of everyday life until they are indistinguishable from it." (Weiser, 1991). He asserts that the future of computing lies in integrating the computer to the real world, not the other way around, and he calls this new paradigm Ubiquitous Computing.

However, the availability of computing resources everytime and everywhere poses the risk of information overload on the user. To solve this problem, the concept of "calm technology" was created (Weiser & Brown, 1996). This means the transfer of information between the computer and the user must happen in the "periphery" of the user's attention, moving to the "center" of his attention in moments of urgency. Taking a car's engine as an example: when a person drives a car, his attention is focused on the road, and he barely notices the engine sound. However, if the engine emits a strange sound (that might indicate some kind of problem), the driver's attention turns to the engine.

If a system is to be minimally intrusive, it must undestand the situation of the user and his surroundings, and adapt its own behaviour according to this information. This information is called context, and systems capable of this kind of adaptation are called context-sensitive (Satyanarayanan, 2001; Dey, 2001).

Research in this area shows that is not only useful to know the context of the user when a service is being offered, but that the history of visited contexts can be a valuable information to help the application to adapt itself to the user. This history of visited contexts is called Trail (Silva et al., 2010). Altough the trail is usually tied to the location of a user, the concept of a trail is breader and includes activities performed, applications used, content accessed – among others – inside a specific context and timeframe (Silva et al., 2009).

For an application to adapt itself effectively to a user, it is not only important to know the history of contexts visited by a user, but to understand who the user is – that is, what are his characteristics. This kind of application organize its knowledge about a user in User Profiles. The profiles are stored in a format called User Model (Viviani et al., 2010).

Personalization means the individual adaptation of products, services and information. Personalization technology usually involve programs that learn patterns, habits and preferences of a user. The main goal of modern personalization systems is to offer the users what they want, without demanding them to inform it explicitly (Mulvenna et al., 2000). It is clear that this pur-

pose meets the ubiquitous computing objective that is, the development of a technology that "disappears".

For a system to be succeessful in obtaining the user's intent, it must know the user, the context where the user is inserted and the actions he took when he was previously in the same context or in similar contexts. Although there are some studies that seek to infer knowledge about a user from their trail, previous works operate within a specific context (such as Ubiquitous Education (Barbosa et al., 2011) or Ubiquitous Commerce (Franco et al., 2011)) or use a static user model (Levis et al., 2008).

A relevant aspect for environments with multiple independent systems is the interoperability. Interoperability is the ability of two or more systems or components to exchange information and use the information that has been exchanged. (IEEE, 1991). In the context of customization, interoperability refers to the ability to access and interpret information derived from multiple heterogeneous sources, and integrate it into a common user model. Interoperability can be (i) syntactic, where the differences between the models are resolved at application level, or (ii) semantic, where the differences between the models are resolved at the level of knowledge (Carmagnola & Cena, 2009).

Research indicates that it is desirable that future systems become increasingly invisible to their users (Weiser & Brown, 1996), and that a system can only become invisible if it is proactive, and can only be proactive by knowing the user (Satyanarayanan, 2001). For a system to know a user, he must know the user history, understand the context in which he is inserted, as well as data collected concerning him in the same context (or in a similar context), and be able to process this set of information into a user profile.

In this context we are proposing the eProfile, a model for the automatic creation of profiles by inference of entity trails. This model allows the use of a single profile by several independent systems, offering the possibility of semantic interoperability between them.

In this model, the concept of entity is used instead of the classic concept of User. While usually the "user" means the person who is using the application, an entity refers to any object (physical or not) able to control an application. Thus, an entity can be a person, a moving object (like a car) or property (like a room), or even a different system.

This article is organized as follows. Section 2 proposes the eProfile model. Section 3 discusses implementation aspects. Section 4 presents an experiment validating the model. Section 5 describes related work, showing the contributions of this model. Finally, a Section 6 presents the concluding remarks.

2 eProfile Model

Figure 1 shows the overview of eProfile, where each circle represents a component, and the arrows indicate the flow of data.

The first component depicted is the **application**. The application, in this model, is any application that is being used directly by an entity. For an

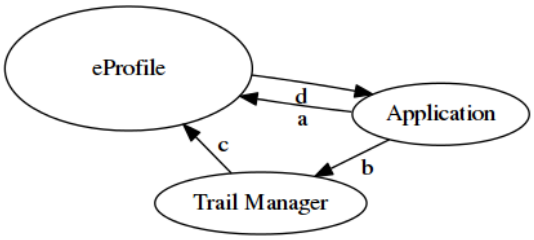

Fig. 1 Overview of eProfile

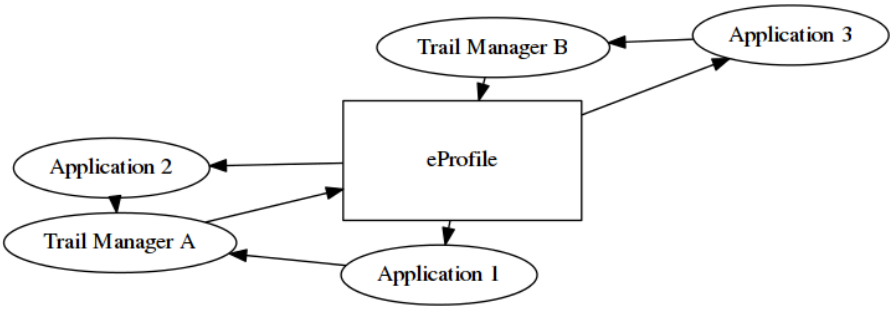

Fig. 2 eProfile connected to multiple trail managers

application to be considered suitable to use this model and the functionality offered by eProfile, it needs to record their information on a trail manager supported by eProfile. The application must also implement a communication protocol to exchange messages with eProfile. If these prerequisites are followed, the application can be of any type and operate in any type of device that has communication with the Internet.

The second component depicted is a **trail manager**. A trail manager is capable of storing events held by an entity, putting them in chronological order and registering information relevant to the event (eg, the context in which the event was held). The eProfile manager supports any trail manager whose communication protocol is implemented in eProfile.

The third component of Figure 1 represents the **eProfile**. The arrows indicate the flow of data. The communication starts when the application registers itself on eProfile, sending (i) the entity model used by the application and (ii) the rules for construction of this model (arrow "a"). From this moment on, the application starts recording the events on the trail manager (arrow "b"). The eProfile periodically retrieves this information from the trail manager and builds or reconfigures the profile entity (arrow "c"). When the application needs an updated profile, it retrieves this profile from eProfile (arrow "d").

Since the eProfile supports multiple applications, it is possible for different applications to use different trail managers. Therefore, the eProfile also supports communication with a number of trail managers, as exemplified in Figure 2.

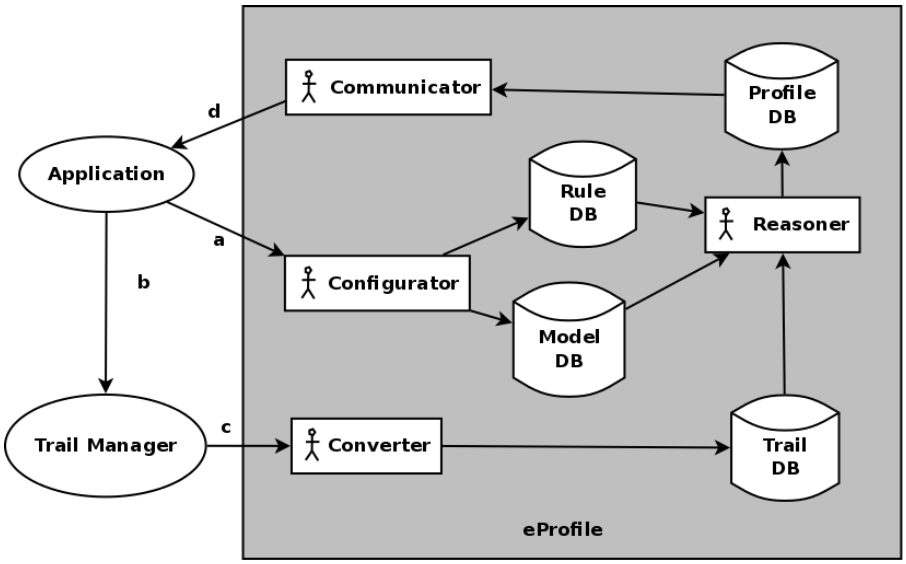

Fig. 3 eProfile Architecture

2.1 Architecture

Figure 3 shows the architecture of eProfile. The components "Application" and "Trail Manager" are the same as shown in Figure 1, while the "eProfile" service was expanded to show its inner workings.

Since the eProfile is a model that infers information about data stored in an ontology autonomously, and is organized into independent parts, it uses the technology of autonomous agents.

Each application builds its own entity model. The eProfile needs three pieces of information to be able to process the profiles: (i) the entity trail, (ii) the entity model used by the application and (iii) the inference rules for the profile. This can be seen in Figure 3, where the agent "Reasoner" receives data from these three databases. The trail of the entity is loaded by the eProfile from the trail manager, while the other two information are sent directly from the application to the eProfile. Thus, the inference rules defined by the application are used to infer the profile of the entity.

The arrows in Figure 3 indicate the direction of the data flow, where it can be noted two entry points and one exit point.

The first entry point (arrow "a") are the information sent from the application to eProfile or, more specifically, to the **Configurator** agent. The following information must be sent by the application:

- the entity model used by the application, which is the format of the entity profile. Once the entity trail is processed, the inferred data is stored and made available for application using this format;

– rules to infer the profile. These rules will be executed at regular intervals in order to update the entity profile. The Configurator receives the rules, validate them and insert them in **rule database**. Along with the rules, is specified the criticality of information. The system uses this information to determine how often the information will be inferred.

The second entry point (arrow "c") allows receiving the entity trails, which are fetched by the eProfile from the trail manager. Since eProfile can connect to different trail managers, (Figure 2), and each trail manager might use one specific format, a conversion between the original format and the format used by eProfile is needed for the trails to be stored in a standardized manner. This work is performed by an agent named **Converter**, which is able to perform the conversion to the eProfile internal trail format.

The entity trail is then stored in a trail ontology called **eOntoTrail**. This ontology contains the basic classes supporting the construction of a trail, and they can be extended according to the specification of the trail manager that is being used.

Once in possession of the tracks, the model and rules, the **Reasoner** agent infers from the trail, generating a entity profile that consists of an extension of a basic profile ontology named **eOntoProfile**. This ontology, just like the trail ontology, contains basic classes from which applications can specialize, using them to store their own information. The application can then connect to the eProfile and get its entity profile, whose data are sent by the **Communicator** agent (arrow "d").

2.2 Ontologies

As mentioned in Section 2.1, eProfile uses two ontologies: a trail ontology and a profile ontology. These are both basic ontologies, extended by subclasses according to the trail format sent by the trail manager (for the trail ontology) or the needs of the application (for the profile ontology).

The trail ontology data (eOntoTrail), shown in Figure **??**, is fed by different supported trail managers. The Converter agent is responsible for receiving data in the trail manager format and converting it to the eOntoTrail. This way, the reasoner has a single ontology on which to reason.

Context-aware applications are interested in knowing **who**, **where**, **when** and **what** (that is, what the user is doing), so they can discover why the situation is occurring (Abowd et al., 1999). Thus, a trail is represented by a set of events characterized by the following main classes: (i) *Entity*, the identity of the entity that is performing the action, which can optionally be linked to a profile, (ii) *Location*, the place where the action is taking place, (iii) *Time*, the date and time the event occurred, (iv) *Participants*, the entities linked to this action, and (v) *Activity*, the action that the entity is performing (or trying to perform). This class is the most comprehensive of the ontology, and will possibly be the most specialized by the trail managers.

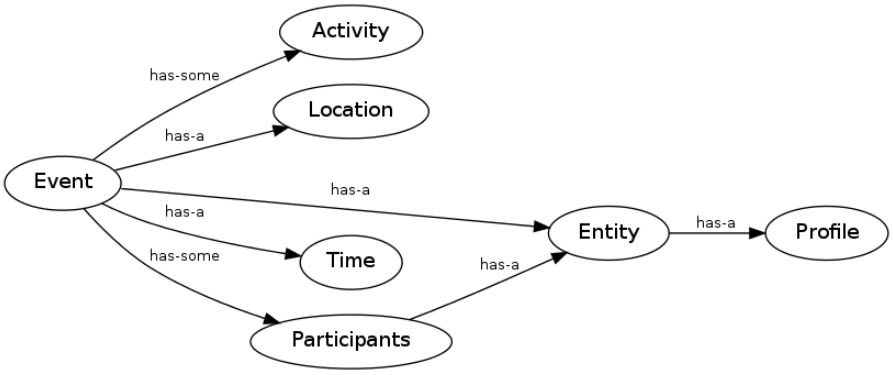

Fig. 4 eProfile Trail Ontology

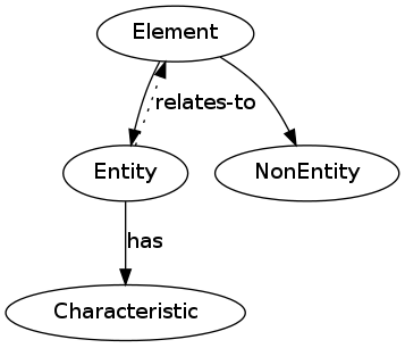

Fig. 5 eProfile Profile Ontology

The profile ontology (eOntoProfile), shown in Figure 5, is the ontology used to store the entity models, as well as the profile information inferred from the trail. For the ontology to be feasible within the proposed model, it must be a generic ontology that allows its classes to be extended by applications. In the figure, a relationship between classes is shown by a dotted line, and a specialization of a class is shown by a continuous line. The eOntoProfile is a basic ontology, extended by the applications for them to assemble their own entity models.

The basic class of the ontology is the *Element* class. The class *Element* represents anything in the real world. It has the following children classes: (i) *Entity*, which represents an entity, that is, an element that has a profile associated with it, and (ii) *NonEntity*, which represents an element than doesn't have a profile associated with it. For example, a trail registers a event in which a person enters a store, but the store has no profile in registered in eProfile. In this case, the store is a *NonEntity*. This distinction is important because in the first case, the eProfile has access to a wide range of information about the entity, entities related to that first entity, and so on.

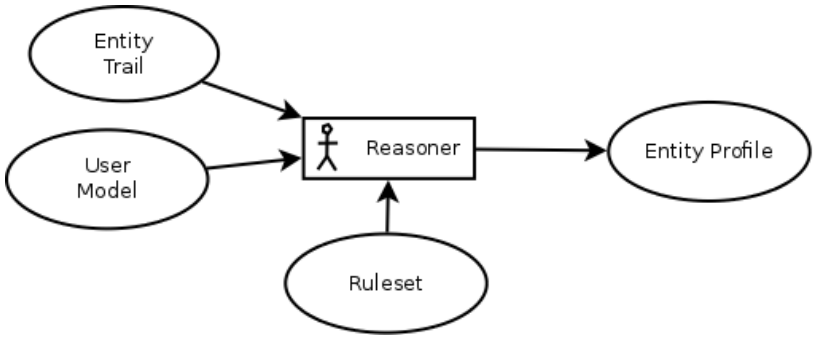

Fig. 6 Data flow for the entity profile inference

The class *Characteristic* represent a aspect of the entity, and is the class that will be extended by applications to refer to any information related to that entity. These characteristics can be simple labels (such as name of a person, or mileage of a car) or more complex information (such as academic knowledge or latest routes traveled).

2.3 Rule Execution

The execution of rules is key to eProfile's operation. A inference rule is a instruction issued by an application, that runs on a entity trailset, resulting in a entity profile according to a entity model previously registered by the application.

For a rule to be executed, the application must have previously registered within eProfile: (i) the rule itself, (ii) in an entity model, which serves as basis for the entity profile format, and (iii) the trail of the entity on which the inference is performed. From these three pieces of information, the reasoner is able to generate a profile entity, according to Figure 6.

The rules are always defined by the application, and a rule is composed of the following set of information: (i) name, (ii) profile field that will be updated, (iii) the rule, written in OWL language (W3C, 2011), (iv) update method (the value can be overwritten or updated according to a formula), (v) update frequency, and (vi) expiration of the rule.

3 Implementation Aspects

The eProfile prototype was developed in Python (Rossum, 1995). Python is a generic, open source, object-oriented, high-level language, that supports multiple architectures. The databases are implemented using the SQLite3 database (Hwaci, 2012). SQLite3 is an SQL transactional self-contained database, available in the public domain. The SQLite3 was chosen due to its good performance and the fact that it does not require a separate server for deployment.

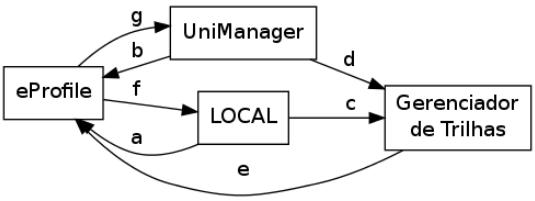

Fig. 7 Scenario model

Figure 6 shows the data flow for the entity profile inference process. At the heart of this operation is the Reasoner agent, which uses an external inference engine to implement the rules. The reasoner used is FACT++, a propositional logic reasoner created as a platform for use of algorithms of semantic *tableaux* and optimization techniques (Tsarkov & Horrocks, 2006). This reasoner was chosen due to its good performance and to support all functions used by the eProfile.

The knowledge representation language used for storage and processing of ontologies is OWLW3C (2011), chosen because for being the most expressive ontology language defined for the Semantic Web (Daconta et al., 2003).

4 Evaluation Aspects

It is only possible to evaluate the functionalities of a proposed infrastructure by building applications that utilize it and then evaluate them, in order to obtain an indirect evaluation of the model (Edwards et al., 2003). The scientific community has been using scenarios to perform this type of indirect evaluation in context sensitive (such as the approach taken by Dey et al. (2001)) and ubiquitous (according to Satyanarayanan (2001)) environments. This work follows the same approach – we created a prototype and a scenario for performing an experiment, in order to evaluate eProfile.

The experiment presented is based on the generation of simulated trails of graduate students within the context of Ubiquitous Learning. Ubiquitous Learning can be understood as a set of educational processes that occur anywhere, at any time and with any device, continuously, contextualized and integrated into the daily lives of apprentices Barbosa et al. (2011).

The scenario is composed of four applications that interact with among themselves, according to Figure 7. LOCAL is a ubiquitous learning system used to find learning opportunities and put students in touch. UniManager is an application used by the administration of the university to manage student grades and disciplines. Both softwares have profile modules, where the profile information is used for decision making. Since in both cases the profiles module is static and depends on collection of explicit information, the eProfile is connected to these softwares, leveraging the dynamic creation of profiles and collecting information implicitly. Furthermore, the eProfile enables interoperability between these systems by allowing information from both systems to

be used to generate single profiles. The scenario also includes a trail managers, that manages the entities trails.

The initial communication occurs between applications and eProfile (arrows "a" and "b"), where applications define their entity models and rules. This communication can occur as often as necessary, because applications can update their rules and their entity models continuously. During the execution of the applications, they record the events in the Trail Manager (arrows "c" and "d"). This means applications can run disconnected from eProfile, and connect only when a rule needs to by updated or when profile data must be downloaded. eProfile retrieves trails periodically from the Trail Manager (arrow "e"). When eProfile is in possession of the trails, rules and entity models, it starts to process these information, inferring new profile data. These profiles are then downloaded by applications (arrows "f" and "g").

The following sections describe the integration between eProfile, LOCAL (Section 4.1) and UniManager (Section 4.2), the simulation environment (Section 4.3), trails, rules and generated profiles (Section 4.4), the simulation parameters (Section 4.5) and results (Seção **??**).

4.1 eProfile/LOCAL Integration

This scenario incorporates the eProfile to a ubiquitous learning system called LOCAL (Location and Context-Aware Learning). LOCAL uses location information and context as an aid to teaching and learning process. A tracking system monitors the mobility of apprentices and, based on their physical position, explores educational opportunities Barbosa et al. (2011).

LOCAL, whose architecture is shown in Figure 8, consists of seven components: (i) a system of user profiles, which stores data relevant to the process of teaching and learning, (ii) a system location, (iii) a Personal Assistant (PA), that follows the user on his mobile device, (iv) a repository of learning objects, (v) a system for sending contextualized messages, (vi) a task scheduling event system and (vii) the tutor, an analysis engine that performs inference using data provided by the profile and tranking systems.

LOCAL's profile system information gathering is explicit. This means that users' (learners) relevant data need to be pre-populated in the system. Furthermore, the LOCAL profile system is static, which means that data is not automatically updated in response to actions taken by learners or external events. Since eProfile is a dynamic entity model with implicit information gathering, integrating these two models results in the enhancement of LOCAL capabilities.

LOCAL fits the eProfile model according to Figure 9. Besides LOCAL and eProfile, 9 presents an application called Trail Manager. A trail manager is an external program that stores the tracks, processes them and forwards them to the eProfile, as described in Section **??**. For this scenario, we created a simple trail manager, which just receives, stores and forwards the trails to the eProfile upon request.

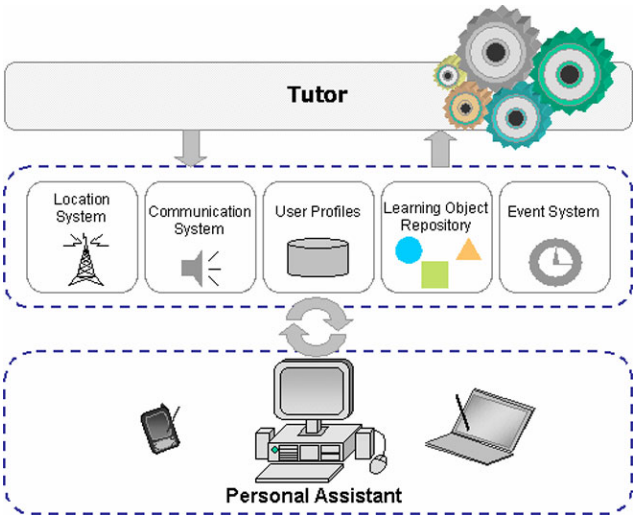

Fig. 8 LOCAL architecture

Integration between eProfile and LOCAL was based on the following changes to the LOCAL: (i) we included a connection interface with eProfile, able to generate learners entity models and rules for profile processing, (ii) we included a connection interface with the trail manager, for the tracking events and actions undertaken on the Personnel Assistant were to be sent to the trail manager, and (iii) we included a communication interface with the eProfile, where the LOCAL User Profiles Module is fed by profiles generated by eProfile.

The first step of integration occurs when the LOCAL communicates with eProfile for the first time. In that momento, LOCAL indicates to the eProfile what is the entity model, and the rules for generating the profiles. This set of information is registeren in eProfile trough the Configurator Agent (arrow " a"), which organizes this information and registers it in the Models and Rules databases.

As learners use LOCAL, their location changes are identified by the Location System module, and their actions are done on the Personal Assistant. These modules record events in the Trail Manager (arrows "b" and "c"), which in turn sends them to the Converter eProfile agent (arrow "d"). Trails sent are stored in the trail database. These trails are periodically processed according to the models and rules defined initially, and a inference process based on this information generates profiles, which are sent to the LOCAL User Profiles module (arrow "e") by the communicator agent. These profiles help LOCAL know its users (learners) in order to distribute content in the most contextualized possible way.

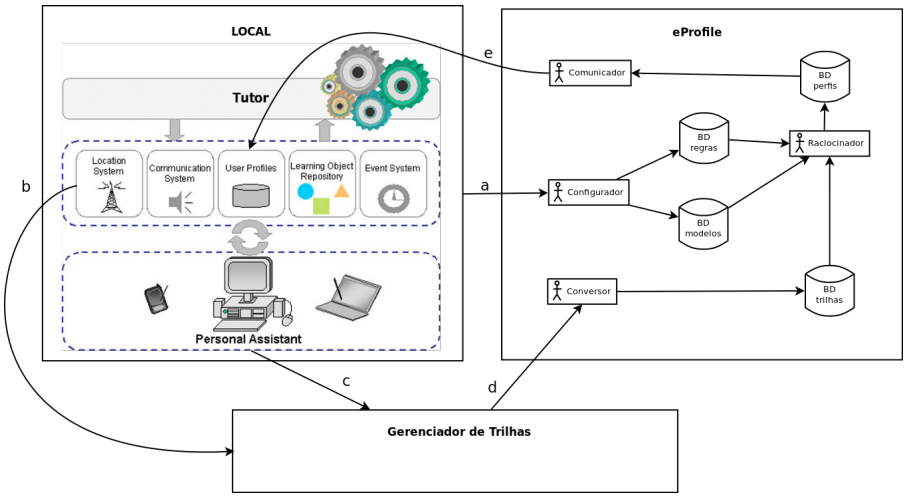

Fig. 9 eProfile/LOCAL Integration

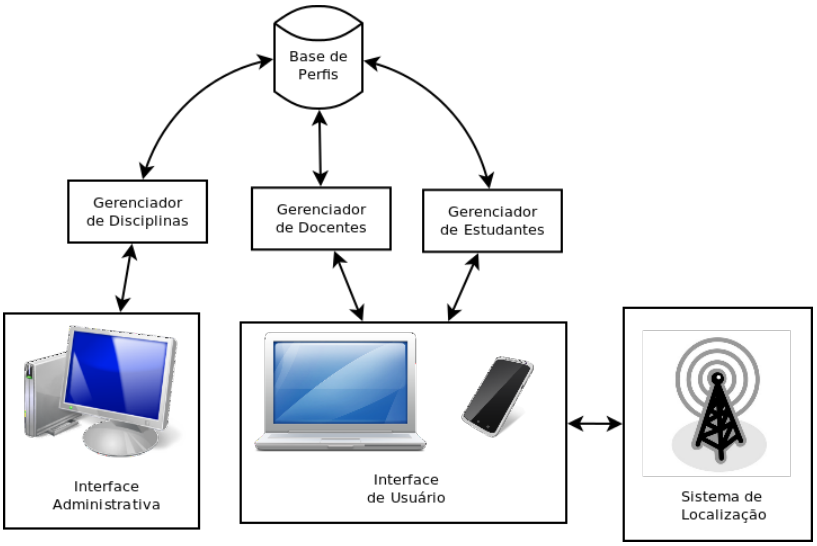

Fig. 10 Modelo do UniManager

4.2 Integração eProfile/UniManager

O UniManager é um *software* administrativo utilizado pela universidade para cadastramento e gerenciamento dos estudantes, docentes e disciplinas. O modelo desta aplicação é exibido na Figura 10.

O UniManager possui duas *interfaces*: administrativa e de usuário. A *interface* administrativa é utilizada pela administração da universidades para cadastrar disciplinas, docentes e estudantes. Ela é utilizada também pelos es-

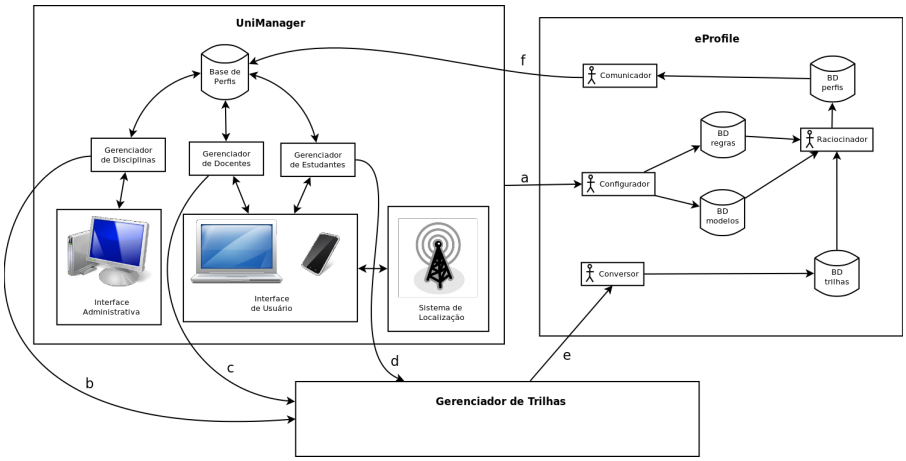

Fig. 11 Integração UniManager/eProfile

tudantes para realizar a sua matrícula para o próximo semestre. Esta *interface* conecta-se a um gerenciador de disciplinas, que gerencia as matrículas e os pré-requisitos entre as disciplinas.

A *interface* de usuário é utilizada nos dispositivos móveis dos professores e dos estudantes, e está conectada a um sistema de localização da Universidade. Desta forma, todos os registros realizados no UniManager a partir desta *interface* contém a localização do aluno dentro da Universidade, isto é, a sala onde o aluno está. O sistema de localização integrado à *interface* de usuário permite verificar se o aluno ou o docente estão presentes em aula. Através desta *interface* o aluno realiza as provas e entrega os trabalhos, e o docente avalia as provas e trabalhos dos alunos.

O UniManager possui também um banco de dados de perfis. Neste banco de dados são armazenadas: (i) informações cadastrais das disciplinas, (ii) presenças e ausências dos professores e alunos e (iii) as provas e disciplinas dos alunos, juntamente com a avaliação feita pelo professor.

A base de perfis é dinâmica, atualizada com base nos eventos registrados, mas está limitada aos dados registrados no próprio UniManager. A integração do UniManager ao eProfile potencializa a geração de perfis mais avançados, utilizando as informações registradas inferidas a partir dos dados gerados pelo LOCAL. Utilizando-se destes dados, é possível registrar na base de perfis do UniManager os interesses do estudante, de modo a poder sugerir a ele disciplinas opcionais que sejam do seu interesse. Além disso, a utilização de trilhas permite que os perfis sejam gerados com base em informações históricas, obtendo informações de perfis que podem ser úteis em análises administrativas (por exemplo, qual combinação de disciplina e professor resultou historicamente na maior nota para os alunos em cada disciplina).

O UniManager se integra ao eProfile de acordo com a Figura 11. No momento da primeira comunicação entre UniManager e eProfile, o UniManager envia um conjunto de mensagens ao eProfile indicando qual serão os modelos

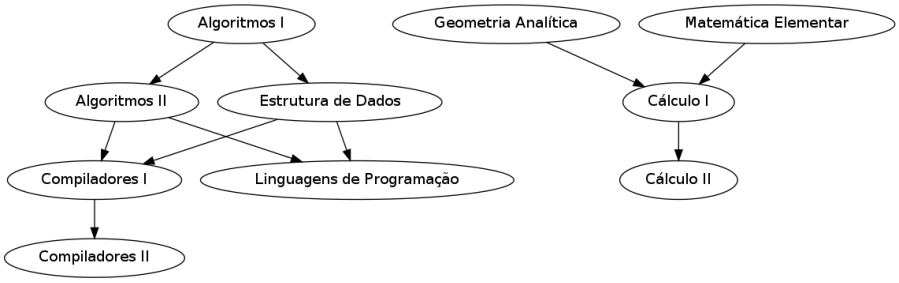

Fig. 12 Pré-requisitos de algumas disciplinas

de entidade esperados e as regras utilizadas para a geração dos perfis (seta "a"). O registro destas informações é feito no agente configurador, responsável por validar e armazenar as regras e os modelos em seus respectivos bancos de dados. À medida que os usuários fazem uso das *interfaces* do UniManager, os dados são processados e registrados pelos módulos gerenciadores de Disciplinas, Docentes e Estudante, que registram no Gerenciador de Trilhas os eventos ocorridos, juntamente com sua localização (setas "b", "c" e "d"). Estas trilhas são organizadas e armazenadas no Gerenciador de Trilhas que, periodicamente, envia as trilhas ao agente Conversor do eProfile (seta "e"). O agente Conversor converte as trilhas para o formato do eProfile e armazena-as no banco de dados de trilhas. Uma vez de posse das trilhas, dos modelos e das regras, o agente raciocinador processa as informações, gerando os perfis que são enviados ao UniManager através do agente comunicador (seta "f").

4.3 Ambiente

O cenário é composto por um conjunto de alunos que realiza um curso de Ciência da Computação. O curso é composto por um conjunto de 40 disciplinas, dentre as quais 12 são opcionais (o aluno deve cumprir ao menos 8 dentre estas 12). Todas as disciplinas possuem outras disciplinas como pré-requisito, exceto pelas disciplinas de início de curso (a Figura 12 exemplifica um subconjunto de disciplinas com seus pré-requisitos). Cada disciplina possui um docente, um número limitado de vagas para os alunos, e um conjunto de recursos (como salas especiais ou equipamentos).

O conjunto de disciplinas utilizado para este cenário é mostrado na Tabela 1, onde as que estão marcadas com um asterisco são opcionais. Algumas requerem um laboratório (com computadores simples), laboratório de redes e laboratório de hardware. As disciplinas sem recursos obrigatórios requerem apenas uma sala de aula.

Os alunos e docentes utilizam dispositivos móveis (*laptop* ou *smartphones*), onde estão instalados aplicativos utilizados para a administração de sua vida acadêmica. Em especial, estão instalados o UniManager e o LOCAL.

Código	Nome	Pré-requisitos	Vagas	Recursos
A1	Algoritmos I	Nenhum	60	Laboratório
A2	Matemática elementar	Nenhum	60	–
A3	Geometria analítica	Nenhum	40	–
A4	Português	Nenhum	80	–
A5	Análise de sistemas I	Nenhum	40	–
B1	Algoritmos II	A1	40	Laboratório
B2	Estrutura de Dados	A1	30	Laboratório
B3	Cálculo I	A2, A3	40	–
B4	Análise de Sistemas II	A5	40	–
B5	Inglês	Nenhum	50	–
C1	Compiladores I	B1, B2	25	Laboratório
C2	Linguagens de Programação	B1, B2	40	Laboratório
C3	Arquitetura de Computadores	Nenhum	10	Laboratório de HW
C4	Engenharia de Software	B4	30	–
C5	Filosofia	A4	30	–
D1	Compiladores II*	C1	20	Laboratório
D2	Microprocessadores*	C3	20	Laboratório de HW
D3	Cálculo II*	B3	30	–
D4	Bancos de Dados	A5	30	Laboratório
D5	Redes*	Nenhum	20	Laboratório de redes
E1	TCC I	C1, C2, C3, C4, D4	15	–
E2	Eletrônica Digital*	D2	10	Laboratório de HW
E3	Inteligência Artificial*	C4, D3	20	–
E4	Computação Gráfica*	C4, D3	20	Laboratório
E5	Gerência de TI*	C4, D4	20	–
F1	TCC II	E1	15	–
F2	Robótica*	E2	10	Laboratório de HW
F3	Sistemas Operacionais*	D1, D2, C4	25	–
F4	Gerência de Projetos*	E5	15	–
F5	Algoritmos Avançados*	B1, B2	15	Laboratório

Table 1 Curso utilizado para o Cenário 1

O UniManager é o sistema utilizado pela universidade para administração das disciplinas, descrito na Seção 4.2. Ele é usado pela administração da universidade, pelos professores e estudantes. A administração utiliza-o para cadastrar disciplinas, alunos e docentes, vincular os alunos e docentes às disciplinas e emitir relatórios gerenciais, como lista de presenças e aprovação/reprovação de alunos, bem como faltas e avaliações de professores. Os docentes utilizam o sistema para realizar as avaliações dos alunos. Os estudantes o utilizam para realizar suas provas e trabalhos, para fazer sua matrícula nas disciplinas e para verificar suas notas e faltas.

Além disso, os docentes e os alunos também utilizam o LOCAL em seus dispositivos móveis. O LOCAL combina as informações dos perfis com sua localização, de modo a poder assisti-los, facilitando a interação entre os aprendizes e permitindo a entrega de material pedagógico contextualizado. Os docentes utilizam o LOCAL para cadastrar o material pedagógico que será distribuído aos alunos. O LOCAL se comunica com os estudantes de modo a distribuir o material pedagógico previamente cadastrado e colocar estudantes com interesses similares em contato durante a realização de atividades pedagógicas. Caso, durante estas atividades, o estudante não tenha nenhum interesse cadastrado

Evento	Horário	Ator	Ação
1	-	Professor	Insere no LOCAL o material pedagógico a respeito de banco de dados, que será utilizado na aula, e requisita a entrega de um trabalho através do UniManager.
2	7:30	Alunos	Nove alunos entram na sala.
3	7:30	LOCAL	LOCAL identifica a entrada dos alunos na sala.
4	7:30	UniManager	UniManager identifica a entrada dos alunos na sala.
5	7:30	LOCAL	LOCAL envia o conteúdo pedagógico da aula aos alunos.
6	8:00	Professor	Professor pede a criação de grupos para o debate.
7	8:00	LOCAL	O LOCAL forma três grupos baseado nas preferências dos estudantes: *Algoritmos Distribuídos* (3 alunos), *Algoritmos de Escalonamento* (3 alunos) e *Algoritmos Matemáricos* (1 aluno). Dois alunos não tem uma preferência cadastrada que permita que o LOCAL os encaixe em algum grupo.
8	8:00	LOCAL	LOCAL solicita aos dois estudantes que escolham algum grupo.
9	8:10	Alunos	Os alunos indicam sua preferência através do Assistente Pessoal do LOCAL.
10	8:10	LOCAL	O LOCAL encaixa os dois alunos nos grupos de sua preferência, um em *Algoritmos de Escalonamento* e o outro em *Algoritmos Matemáticos*.
11	8:10	Alunos	O debate inicia.
12	8:40	Alunos	Um aluno entra na aula, atrasado. O UniManager e o LOCAL registram sua entrada, mas o LOCAL não o encaixa no debate, que já está no final.
13	8:50	Alunos	O debate se encerra.
14	8:50	Professor	O professor pede que os alunos façam um trabalho em aula sobre o debate realizado.
15	10:00	Alunos	Os alunos entregam o trabalho através do UniManager e saem da sala.
16	10:00	UniManager	Recebe os trabalhos entregues e registra a saída dos alunos.
17	10:00	LOCAL	Registra a saída dos alunos.
18	-	Professor	O professor, posteriormente, avalia o trabalho dos alunos e registra a nota no UniManager.

Table 2 Exemplo de uma aula

que corresponda àqueles requeridos pela atividade, ele tem a opção de fazer a escolha utilizando o Assistente Pessoal.

[inline]Jorge, sei que essa última característica não fazia parte do design original do LOCAL, mas você acha que é possível acrescentarmos essa funcionalidade?

A Tabela 2 exemplifica uma aula de duas horas e meia envolvendo 10 alunos, na qual foi realizado um debate a respeito de Técnicas de Programação. Após o debate, os alunos realizaram um trabalho individual a respeito dos assuntos discutidos no debate. Neste exemplo, foram registrados no UniManager a presença dos alunos, a entrega do seu trabalho e a nota obtida e, no LOCAL, o conteúdo pedagógico da aula.

Além disso, o LOCAL registrou a preferência pessoal de dois alunos, a respeito de sistemas de banco de dados. Caso o LOCAL e o UniManager estivessem sendo usados isoladamente, esta informação teria sido apenas relevante para a tarefa imediata da aula. No entanto, com a integração destes sistemas com o eProfile, esta informação é utilizada na atualização do perfil dos alunos, e pode ser utilizada futuramente para facilitar a interação entre alunos e personalizar a entrega de conteúdo pedagógico (pelo LOCAL) e auxiliar o aluno na sugestão de disciplinas facultativas (pelo UniManager).

4.4 Trilhas, Regras e Perfis

Segundo descrito nas Seções 4.1 e 4.2, os aplicativos LOCAL e UniManager sofreram três alterações significativas: (i) foram incluídas *interfaces* de comunicação com o eProfile, para que os aplicativos fornecessem os modelos de entidade e as regras, (ii) foram incluídas *interfaces* de comunicação com o Gerenciador de Trilhas, de modo que os aplicativos pudessem informar os eventos ocorridos e (iii) o módulo de perfis de ambas as aplicações foi modificado de modo a solicitar as atualizações de perfil a partir do eProfile.

O cenário é composto por três entidades: (i) estudantes, (ii) docentes e (iii) disciplinas. As entidades estudante e docente representam, respectivamente, cada um dos alunos e professores envolvidos no curso. A entidade disciplina, que refere-se a cada uma das disciplinas do curso, poderia ser representada tanto como uma entidade quanto como um simples recurso. Neste cenário, foi decidido que ela seria representada como uma entidade, pois desta forma é possível permitir que ela crie uma trilha, da qual pode ser obtido um perfil. Desta forma, o perfil da disciplina pode conter campos que são atualizados repetidamente, tal como a nota média da disciplina.

Segundo descrito na Seção **??**, aplicações sensíveis a contexto estão interessadas em saber **quem**, **onde**, **quando** e **o quê** (isto é, o que o usuário está fazendo), de forma a poderem descobrir por que a situação está ocorrendo. A ontologia de trilha do eProfile, mostrada na Figura 4, define que estas questões são respondidas, respectivamente, pelas classes *Identity*, *Location*, *Time* e *Activity*. Além disso, a classe *Participants* descreve as outras entidades envolvidas neste evento. Quando uma trilha é armazenada no eProfile, a informação referente ao evento deve ser herdada destas classes, utilizando tantas classes quantas forem relevantes para que a informação esteja completa.

A trilha dos estudantes contém os seguintes eventos: cadastro, matrícula em disciplina, entrada e saída em aula, realização de provas e trabalhos, escolha de preferências e abandono de curso. O formato da trilha está detalhado na Figura 13. Na trilha dos docentes são registrados os seguintes eventos: cadastro, entrada e saída em aula e avaliação das provas e trabalhos dos alunos, segundo detalhando na Figura 14. A trilha da disciplina recebe os seguintes eventos: cadastro e início e fim de semestre, de acordo com a Figura 15.

O objetivo final da geração das trilhas é a geração dos perfis. Para que o eProfile tenha sucesso em gerar os perfis a partir das trilhas, ele necessita que o aplicativo informe a ele os modelos de entidades e as regras de inferência.

Os modelos de entidades são a estrutura de dados utilizada para armazenar o perfil de uma entidade. O modelo de entidade utilizado pelo módulo de Perfis de Usuários do LOCAL é mostrado na Figura 16. Os perfis dos aprendizes apresentam dois tipos de dados: (i) persistentes, relacionados ao aprendiz em todas as situações que ele se encontre e (ii) contextuais, associado com os contextos com as quais o aprendiz interage.

As seções do bloco persistente do perfil são (i) contato, que armazena dados básicos do usuário, tais como nome, telefone, entre outros, (ii) preferências, que guarda o tipo de mídia preferida pelo usuário e (iii) interesses, que identifica

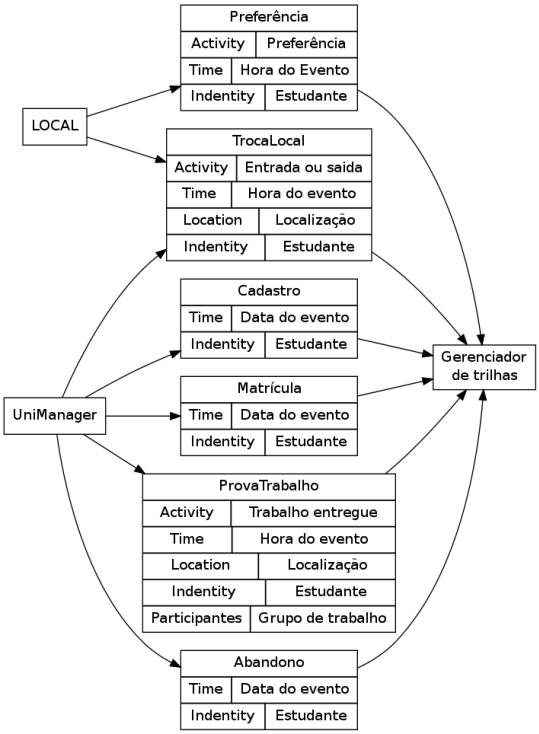

Fig. 13 Ontologia de trilha do estudante

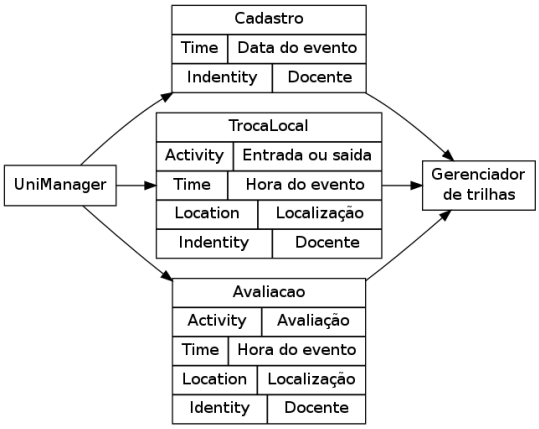

Fig. 14 Ontologia de trilha do docente

tópicos de interesse dos alunos. A seção de interesses está categorizada de acordo com o sistema de classificação computacional da ACM **?**. As seções do bloco contextual são (i) relações, que identifica os relacionamentos com outros usuários dentro do mesmo contexto, (ii) performance, que diz respeito

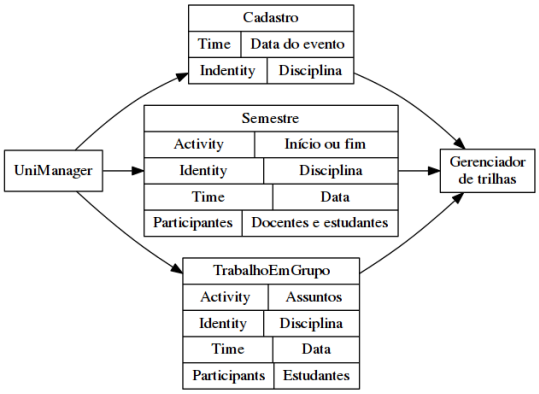

Fig. 15 Ontologia de disciplina

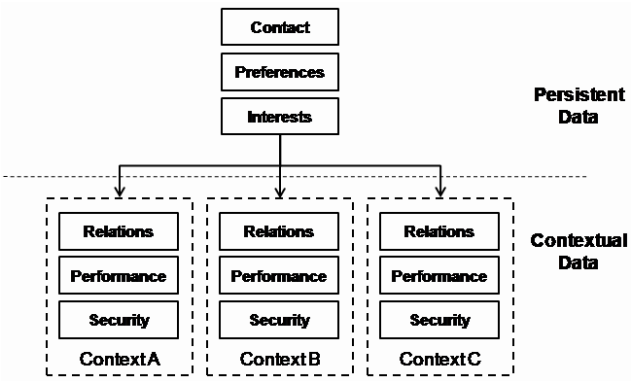

Fig. 16 Modelo de perfis do LOCAL

a objetivos que devem ser atingidos pelos estudantes e (iii) segurança, que armazena credenciais necessárias para acessar diferentes níveis em contextos específicos.

O UniManager armazena perfis de disciplinas, docentes e estudantes, e tem seu modelo de entidade ilustrado na Figura 17. As informações básicas, disciplinas matriculadas e notas/faltas do perfil do estudante são utilizadas para atividades gerenciais da Universidade, como verificar se o aluno pode cursar uma determinada disciplina, assim como as informações de disciplinas do docente. As informações de desempenho do docente e de médias históricas das disciplinas são utilizadas para ajustar o docente com a disciplina que tenha maior afinidade ou desempenho. As informações de interesse dos alunos são utilizadas para sugerir disciplinas opcionais aos alunos.

O UniManager e o LOCAL definem um conjunto de regras para geração dos perfis baseados nas trilhas geradas por ambos os aplicativos. Um aplicativo tem a possibilidade de utilizar a trilha de outro aplicativo para o processamento

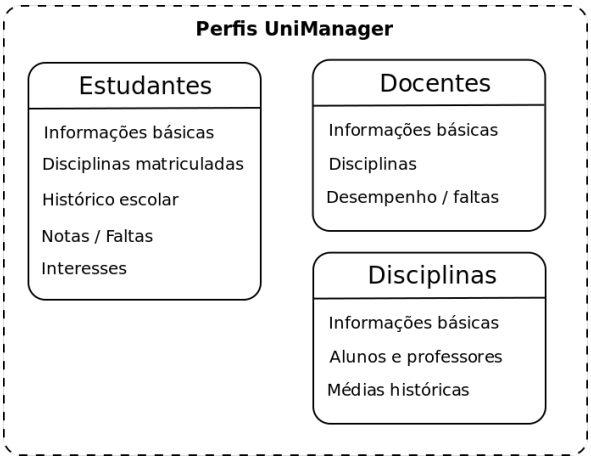

Fig. 17 Modelo de perfis do UniManager

das regras e consequente geração dos perfis, permitido assim que o perfil da entidade seja enriquecido pelo ambiente que o cerca.

A definição das regras é feita em OWL, através da criação de classes que possam ser processadas por um raciocinador. A Tabela 2 mostra o exemplo de uma aula onde o professor cadastra no LOCAL um conjunto de assuntos que serão debatidos em aula (evento 1). Este evento gera uma trilha que é enviada do LOCAL ao Gerenciador de Trilhas e, subsequentemente, ao eProfile.

O LOCAL, então, tenta identificar os alunos que possuam iteresses iguais aos do debate proposto, ou interesses que estejam relacionadas a estes (evento 7). Desta forma, o LOCAL envia as regras OWL descritas na Figura 18 para realizar esta identificação. A primeira regra chama-se *RelatedSubjects* (linha 2), onde são inferidos todos os assuntos (classe *Subject*, linha 4) que sejam relacionados (linha 6) aos assuntos que serão debatidos em aula (linha 7). A expressão `?subject` é uma extensão do eProfile, que substitui esta expressão pelos assuntos que estão sendo utilizados, permitindo que a mesma regra possa ser usada em diferentes situações. Esta regra, após inferida, resulta na listagem de todos os assuntos (classes do tipo *Subject*) que se relacionam aos assuntos que serão debatidos, colocando estas classes como subclasses da classe *RelatedSubjects*.

A partir desta regra, é possível buscar os estudantes (linha 15) cujos interesses se intersectam com os interesses encontrados na classe *RelatedSubjects* (linha 18). Estes estudantes, na ontologia, passam a ser subclasses da classe *StudentsSelected* (linha 13).

O evento 7 da Tabela 2 descreve que dois alunos não se encaixaram em nenhum dos grupos pois não tinham nenhum interesse cadastrado relacionado aos assuntos do debate, e o LOCAL questionou-os em qual assunto eles gostariam de se encaixar (eventos 8 e 9), gerando assim um evento de trilha para estes dois estudantes. Uma vez que o registro tenha sido feito, o perfil dos estudantes é reprocessado de acordo com as regras definidas pelo LOCAL.

```
1  <EquivalentClasses>
2    <Class IRI="RelatedSubjects"/>
3    <ObjectIntersectionOf>
4      <Class IRI="Subject"/>
5      <ObjectSomeValuesFrom>
6        <ObjectProperty IRI="isRelated"/>
7        <Class IRI="?subject"/>
8      </ObjectSomeValuesFrom>
9    </ObjectIntersectionOf>
10 </EquivalentClasses>
11
12 <EquivalentClasses>
13   <Class IRI="StudentsSelected"/>
14   <ObjectIntersectionOf>
15     <Class IRI="Student"/>
16     <ObjectSomeValuesFrom>
17       <ObjectProperty IRI="hasInterest"/>
18       <Class IRI="RelatedSubjects"/>
19     </ObjectSomeValuesFrom>
20   </ObjectIntersectionOf>
21 </EquivalentClasses>
```

Fig. 18 Regra OWL para identificação de interesses dos alunos

```
1 Profile:   Student
2 Field:     Interest
3 Rule:      StudentInterest
4 Update:    addition
5 Frequence: onTrailEvent(SubjectChosen)
6 Expires:   never
```

Fig. 19 Regra para processamento do perfil do estudante

O cabeçalho da regra que atualiza o campo de interesses de um aluno é mostrado na Figura 19. Este cabeçalho define que o campo *Interest* (linha 2) dos perfis dos estudantes (linha 1) serão atualizados com as subclasses da classe *StudentInterest*, cuja regra OWL é mostrada na Figura 20. Também é definido que um novo interesse é somado ao interesses que já existem (linha 4), que esta regra é atualizada toda vez que é registrado um evento de trilha na qual um assunto é escolhido (linha 5). A regra OWL da Figura 20 define que a classe *StudentInterest* é formada dos assuntos escolhidos (linha 6) pelos estudantes que fazem parte deste contexto (linha 10 e 11) e que tenham escolhido um determinado assunto (linhas 6 e 7).

Após a aplicação da regra mostrada nas Figuras 19 e 20, a regra da Figura 18 é executada novamente, desta vez identificando os alunos que fizeram sua escolha (evento 10 da Tabela 2.

4.5 Protótipo e Simulação

O protótipo do eProfile foi desenvolvido na linguagem Python Rossum (1995). Ele utiliza o HermiT como motor de inferência **?**. A comunicação entre estes

```
<EquivalentClasses>
  <Class IRI="StudentInterest"/>
  <ObjectIntersectionOf>
    <Class IRI="SubjectChosen"/>
    <ObjectSomeValuesFrom>
      <ObjectProperty IRI="subjectChosen"/>
      <Class IRI="Subject"/>
    </ObjectSomeValuesFrom>
    <ObjectSomeValuesFrom>
      <ObjectProperty IRI="hasStudent"/>
      <Class IRI="?student"/>
    </ObjectSomeValuesFrom>
  </ObjectIntersectionOf>
</EquivalentClasses>
```

Fig. 20 Regra OWL para processamento de perfil de estudante

dois aplicativos se dá via troca de arquivos: quando é necessário que seja feito o processamento de algum perfil, o eProfile gera um arquivo OWL contendo as regras de processamento e chama o HermiT, que processa a ontologia e salva um novo arquivo com a ontologia gerada. O eProfile lê então este arquivo para dar prosseguimento ao processo.

Para a simulação do modelo, foram desenvolvidos dois programas:

1. Um simulador, que une as funcionalidades do UniManager e do LOCAL gerando trilhas simuladas dos estudantes, dos docentes e das disciplinas;
2. Um gerenciador de trilhas simulado, que armazena as trilhas geradas e disponibiliza-as para o eProfile.

A comunicação entre estes três aplicativos (os dois aplicativos simulados e o eProfile) é feita através de *webservices* utilizando REST **?**.

Para simular o cenário de forma mais fiel possível, cada uma das entidades é criada com um conjunto de características que as distinguem umas das outras, e faz com que tenham desempenhos diferentes e tomem decisões diferentes quando confrontadas com uma escolha. Os docentes são gerados com valores aleatórios de experiência e rigor. Estas características influem diretamente no nível de aprendizado dos estudantes, na dificuldade da disciplina e na probabilidade de desistência dos alunos. Os estudantes são gerados com níveis variados de empenho, inteligência e gostos pessoais. Estas características influenciam o desempenho do aluno, a probabilidade de desistência das disciplinas e do curso, e as escolhas que o aluno irá realizar.

O cenário simula um conjunto de 40 alunos que realiza um curso de ciência da computação em uma Universidade. O tempo da simulação é aberto, sendo que a mesma será executada até que todos os alunos se formem ou tenham desistido do curso.

Uma simulação inicia com a criação das entidades de disciplinas, estudantes e docentes. São então gerados os perfis iniciais das disciplinas, e os docentes são distribuídos aleatoriamente entre as disciplinas. A partir daí, inicia-se um *loop* que termina apenas quando todos os alunos tiverem concluído ou abandonado

o curso. Cada iteração do *loop* representa um semestre do curso contém os seguintes passos:

1. Os alunos escolhem as disciplinas que irão cursar e se matriculam;
2. Cada dia da disciplina é simulado, onde os alunos assistem a aula e realizam as provas ou entregam os trabalhos;
3. A nota final dos alunos é calculada, e os alunos decidem se continuarão ou abandonarão o curso.

Cada disciplina é composta por 14 aulas. Existem dois tipos de eventos que podem ocorrer em uma aula: (i) um trabalho ou debate em grupo, e (ii) um trabalho ou prova individual, sendo que uma aula pode ter um dos dois eventos, ambos ou nenhum. A Tabela 2 exemplifica uma aula em que os dois tipos de eventos ocorreram.

5 Related Works

...

6 Conclusion

...

References

Abowd, Gregory D., Anind K. Dey, Peter J. Brown, Nigel Davies, Mark Smith, & Pete Steggles 1999. Towards a Better Understanding of Context and Context-Awareness. In Proceedings of the 1st international symposium on Handheld and Ubiquitous Computing, HUC '99, pages 304–307, London, UK. Springer-Verlag.

Barbosa, Jorge Luis Victoria, Rodrigo Machado Hahn, Debora Nice Ferrari Barbosa, & Amarolinda Iara da Costa Zanel Saccol 2011. A ubiquitous learning model focused on learner interaction. International Journal of Learning Technology 2011, 6(1):62 – 83.

Carmagnola, Francesca, & Federica Cena 2009. User identification for cross-system personalisation. Inf. Sci., 179:16–32.

Daconta, Michael C., Leo J. Obrst, & Kevin T. Smith 2003. The Semantic Web : A Guide to the Future of XML, Web Services, and Knowledge Management. Wiley, Indianapolis.

Dey, A. 2001. Understanding and Using Context. Personal and Ubiquitous Computing, 6(1):4–7.

Dey, Anind K., Gregory D. Abowd, & Daniel Salber 2001. A conceptual framework and a toolkit for supporting the rapid prototyping of context-aware applications. Hum.-Comput. Interact., 16(2):97–166.

Edwards, W. Keith, Victoria Bellotti, Anind K. Dey, & Mark W. Newman 2003. The challenges of user-centered design and evaluation for infrastructure. In Proceedings of the SIGCHI conference on Human factors in computing systems, CHI '03, pages 297–304, New York, NY, USA. ACM.

Franco, Laerte K., Joao H. Rosa, Jorge L.V. Barbosa, Cristiano A. Costa, & Adenauer C. Yamin 2011. MUCS: A model for ubiquitous commerce support. Electronic Commerce Research and Applications, 10(2):237 – 246. ¡ce:title¿Special Issue on Electronic Auctions: Strategies and Methods¡/ce:title¿.

Heckmann, Dominik 2005. Ubiquitous User Modeling. PhD thesis, Department of Computer Science, Saarland University, Germany.

Hwaci 2012. SQLite Home Page. `http://www.sqlite.org/`. Accessed in 05/05/2012.

IEEE 1991. IEEE Standard Computer Dictionary. A Compilation of IEEE Standard Computer Glossaries. IEEE Std 610, page 1.

Levis, Darci, Jorge L. V. Barbosa, Sérgio Crespo C. S. Pinto, & Debora Nice Ferrari Barbosa 2008. Aperfeiçoamento Automático do Perfil do Aprendiz em Ambientes de Educação Ubíqua. Revista Brasileira de Informática na Educação, 16(1).

Mulvenna, Maurice D., Sarabjot S. Anand, & Alex G. Büchner 2000. Personalization on the Net using Web mining: introduction. Commun. ACM, 43:122–125.

Rossum, Guido 1995. Python reference manual. Technical report, Amsterdam, The Netherlands, The Netherlands.

Satyanarayanan, M. 2001. Pervasive Computing: Vision and Challenges. IEEE Personal Communications, 8:10–17.

Silva, Jader, João Rosa, Jorge Barbosa, Débora Barbosa, & Luiz Palazzo 2010. Content distribution in trail-aware environments. Journal of the Brazilian Computer Society, 16:163–176. 10.1007/s13173-010-0015-1.

Silva, Jader, João Rosa, Jorge Barbosa, Débora N. F. Barbosa, & Luiz Antônio Moro Palazzo 2009. Content distribution in trail-aware environments. In Proceedings of the XV Brazilian Symposium on Multimedia and the Web, WebMedia '09, pages 15:1–15:8, New York, NY, USA. ACM.

Tsarkov, D., & I. Horrocks 2006. FaCT++ Description Logic Reasoner: System Description. In Proc. of the Int. Joint Conf. on Automated Reasoning (IJCAR 2006), volume 4130 of *Lecture Notes in Artificial Intelligence*, pages 292–297. Springer.

Viviani, Marco, Nadia Bennani, & Elod Egyed-Zsigmond 2010. A Survey on User Modeling in Multi-Application Environments. In The Third International Conference on Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services CENTRIC'10, pages 111–116. IEEE.

W3C 2011. OWL Web Ontology Language Guide. `http://www.w3.org/TR/owl-guide/`. Accessed in 05/05/2012.

Weiser, Mark 1991. The Computer for the 21st Century. Scientific America, pages 94–104.

Weiser, Mark, & John Seely Brown 1996. Designing Calm Technology. PowerGrid Journal.